

Pillar 1: Core Pages

Only the sections you need to implement Assignment 1 · 17 pages

THE MODERN ARCHITECTURE BLUEPRINT

LLaMA: Open and Efficient Foundation Language Models

Touvron et al., 2023 · arXiv:2302.13971 · original pp. 1–3

Included: Introduction and Approach: pre-normalization with RMSNorm, SwiGLU, rotary embeddings, AdamW settings, and Table 2 (model sizes and hyperparameters).

p. 2
3 pp

POSITIONAL EMBEDDINGS

RoFormer: Enhanced Transformer with Rotary Position Embedding

Su et al., 2021 · arXiv:2104.09864 · original pp. 4–7

Included: Section 3: formulation, 3.2 RoPE (the 2D rotation case and the general form), properties, derivation, and the efficient element-wise implementation (Eq. 34).

p. 5
4 pp

NORMALIZATION

Root Mean Square Layer Normalization

Zhang & Sennrich, 2019 · arXiv:1910.07467 · original pp. 3–5

Included: LayerNorm background, the RMSNorm definition, pRMSNorm, and the invariance and gradient analysis.

p. 9
3 pp

ACTIVATIONS

GLU Variants Improve Transformer

Shazeer, 2020 · arXiv:2002.05202 · original pp. 1–2

Included: The Transformer feed-forward layer and the GLU variant definitions, including SwiGLU and $\text{FFN}_{\text{SwiGLU}}$.

p. 12
2 pp

TOKENIZATION

Neural Machine Translation of Rare Words with Subword Units

Sennrich et al., 2015 · arXiv:1508.07909 · original pp. 3–4

Included: Section 3.2 Byte Pair Encoding and Algorithm 1 (the merge loop, in Python).

p. 14
2 pp

OPTIMIZATION

Decoupled Weight Decay Regularization

Loshchilov & Hutter, 2017 · arXiv:1711.05101 · original pp. 2–3

Included: Section 2: why L_2 regularization is not weight decay for Adam (Propositions 1–2), and Algorithm 2 (Adam with L_2 vs. AdamW).

p. 16
2 pp

LLaMA: Open and Efficient Foundation Language Models

Hugo Touvron*, Thibaut Lavril*, Gautier Izacard*, Xavier Martinet
Marie-Anne Lachaux, Timothee Lacroix, Baptiste Rozière, Naman Goyal
Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin
Edouard Grave*, Guillaume Lample*

Meta AI

Abstract

We introduce LLaMA, a collection of foundation language models ranging from 7B to 65B parameters. We train our models on trillions of tokens, and show that it is possible to train state-of-the-art models using publicly available datasets exclusively, without resorting to proprietary and inaccessible datasets. In particular, LLaMA-13B outperforms GPT-3 (175B) on most benchmarks, and LLaMA-65B is competitive with the best models, Chinchilla-70B and PaLM-540B. We release all our models to the research community¹.

1 Introduction

Large Languages Models (LLMs) trained on massive corpora of texts have shown their ability to perform new tasks from textual instructions or from a few examples (Brown et al., 2020). These few-shot properties first appeared when scaling models to a sufficient size (Kaplan et al., 2020), resulting in a line of work that focuses on further scaling these models (Chowdhery et al., 2022; Rae et al., 2021). These efforts are based on the assumption that more parameters will lead to better performance. However, recent work from Hoffmann et al. (2022) shows that, for a given compute budget, the best performances are not achieved by the largest models, but by smaller models trained on more data.

The objective of the scaling laws from Hoffmann et al. (2022) is to determine how to best scale the dataset and model sizes for a particular *training* compute budget. However, this objective disregards the *inference* budget, which becomes critical when serving a language model at scale. In this context, given a target level of performance, the preferred model is not the fastest to train but the fastest at inference, and although it may be cheaper to train a large model to reach a certain level of

performance, a smaller one trained longer will ultimately be cheaper at inference. For instance, although Hoffmann et al. (2022) recommends training a 10B model on 200B tokens, we find that the performance of a 7B model continues to improve even after 1T tokens.

The focus of this work is to train a series of language models that achieve the best possible performance at various inference budgets, by training on more tokens than what is typically used. The resulting models, called *LLaMA*, ranges from 7B to 65B parameters with competitive performance compared to the best existing LLMs. For instance, LLaMA-13B outperforms GPT-3 on most benchmarks, despite being $10\times$ smaller. We believe that this model will help democratize the access and study of LLMs, since it can be run on a single GPU. At the higher-end of the scale, our 65B-parameter model is also competitive with the best large language models such as Chinchilla or PaLM-540B.

Unlike Chinchilla, PaLM, or GPT-3, we only use publicly available data, making our work compatible with open-sourcing, while most existing models rely on data which is either not publicly available or undocumented (e.g. “Books – 2TB” or “Social media conversations”). There exist some exceptions, notably OPT (Zhang et al., 2022), GPT-NeoX (Black et al., 2022), BLOOM (Scao et al., 2022) and GLM (Zeng et al., 2022), but none that are competitive with PaLM-62B or Chinchilla.

In the rest of this paper, we present an overview of the modifications we made to the transformer architecture (Vaswani et al., 2017), as well as our training method. We then report the performance of our models and compare with others LLMs on a set of standard benchmarks. Finally, we expose some of the biases and toxicity encoded in our models, using some of the most recent benchmarks from the responsible AI community.

* Equal contribution. Correspondence: {htouvron, thibautlav, gizacard, egrave, glample}@meta.com

¹<https://github.com/facebookresearch/llama>

2 Approach

Our training approach is similar to the methods described in previous work (Brown et al., 2020; Chowdhery et al., 2022), and is inspired by the Chinchilla scaling laws (Hoffmann et al., 2022). We train large transformers on a large quantity of textual data using a standard optimizer.

2.1 Pre-training Data

Our training dataset is a mixture of several sources, reported in Table 1, that cover a diverse set of domains. For the most part, we reuse data sources that have been leveraged to train other LLMs, with the restriction of only using data that is publicly available, and compatible with open sourcing. This leads to the following mixture of data and the percentage they represent in the training set:

English CommonCrawl [67%]. We preprocess five CommonCrawl dumps, ranging from 2017 to 2020, with the CCNet pipeline (Wenzek et al., 2020). This process deduplicates the data at the line level, performs language identification with a fastText linear classifier to remove non-English pages and filters low quality content with an n-gram language model. In addition, we trained a linear model to classify pages used as references in Wikipedia v.s. randomly sampled pages, and discarded pages not classified as references.

C4 [15%]. During exploratory experiments, we observed that using diverse pre-processed CommonCrawl datasets improves performance. We thus included the publicly available C4 dataset (Raffel et al., 2020) in our data. The preprocessing of C4 also contains deduplication and language identification steps: the main difference with CCNet is the quality filtering, which mostly relies on heuristics such as presence of punctuation marks or the number of words and sentences in a webpage.

Github [4.5%]. We use the public GitHub dataset available on Google BigQuery. We only kept projects that are distributed under the Apache, BSD and MIT licenses. Additionally, we filtered low quality files with heuristics based on the line length or proportion of alphanumeric characters, and removed boilerplate, such as headers, with regular expressions. Finally, we deduplicate the resulting dataset at the file level, with exact matches.

Wikipedia [4.5%]. We add Wikipedia dumps from the June-August 2022 period, covering 20

Dataset	Sampling prop.	Epochs	Disk size
CommonCrawl	67.0%	1.10	3.3 TB
C4	15.0%	1.06	783 GB
Github	4.5%	0.64	328 GB
Wikipedia	4.5%	2.45	83 GB
Books	4.5%	2.23	85 GB
ArXiv	2.5%	1.06	92 GB
StackExchange	2.0%	1.03	78 GB

Table 1: **Pre-training data.** Data mixtures used for pre-training, for each subset we list the sampling proportion, number of epochs performed on the subset when training on 1.4T tokens, and disk size. The pre-training runs on 1T tokens have the same sampling proportion.

languages, which use either the Latin or Cyrillic scripts: bg, ca, cs, da, de, en, es, fr, hr, hu, it, nl, pl, pt, ro, ru, sl, sr, sv, uk. We process the data to remove hyperlinks, comments and other formatting boilerplate.

Gutenberg and Books3 [4.5%]. We include two book corpora in our training dataset: the Gutenberg Project, which contains books that are in the public domain, and the Books3 section of ThePile (Gao et al., 2020), a publicly available dataset for training large language models. We perform deduplication at the book level, removing books with more than 90% content overlap.

ArXiv [2.5%]. We process arXiv Latex files to add scientific data to our dataset. Following Lewkowycz et al. (2022), we removed everything before the first section, as well as the bibliography. We also removed the comments from the .tex files, and inline-expanded definitions and macros written by users to increase consistency across papers.

Stack Exchange [2%]. We include a dump of Stack Exchange, a website of high quality questions and answers that covers a diverse set of domains, ranging from computer science to chemistry. We kept the data from the 28 largest websites, removed the HTML tags from text and sorted the answers by score (from highest to lowest).

Tokenizer. We tokenize the data with the byte-pair encoding (BPE) algorithm (Sennrich et al., 2015), using the implementation from SentencePiece (Kudo and Richardson, 2018). Notably, we split all numbers into individual digits, and fallback to bytes to decompose unknown UTF-8 characters.

params	dimension	n heads	n layers	learning rate	batch size	n tokens
6.7B	4096	32	32	$3.0e^{-4}$	4M	1.0T
13.0B	5120	40	40	$3.0e^{-4}$	4M	1.0T
32.5B	6656	52	60	$1.5e^{-4}$	4M	1.4T
65.2B	8192	64	80	$1.5e^{-4}$	4M	1.4T

Table 2: Model sizes, architectures, and optimization hyper-parameters.

Overall, our entire training dataset contains roughly 1.4T tokens after tokenization. For most of our training data, each token is used only once during training, with the exception of the Wikipedia and Books domains, over which we perform approximately two epochs.

2.2 Architecture

Following recent work on large language models, our network is based on the transformer architecture (Vaswani et al., 2017). We leverage various improvements that were subsequently proposed, and used in different models such as PaLM. Here are the main difference with the original architecture, and where we were found the inspiration for this change (in bracket):

Pre-normalization [GPT3]. To improve the training stability, we normalize the input of each transformer sub-layer, instead of normalizing the output. We use the RMSNorm normalizing function, introduced by Zhang and Sennrich (2019).

SwiGLU activation function [PaLM]. We replace the ReLU non-linearity by the SwiGLU activation function, introduced by Shazeer (2020) to improve the performance. We use a dimension of $\frac{2}{3}4d$ instead of $4d$ as in PaLM.

Rotary Embeddings [GPTNeo]. We remove the absolute positional embeddings, and instead, add rotary positional embeddings (RoPE), introduced by Su et al. (2021), at each layer of the network.

The details of the hyper-parameters for our different models are given in Table 2.

2.3 Optimizer

Our models are trained using the AdamW optimizer (Loshchilov and Hutter, 2017), with the following hyper-parameters: $\beta_1 = 0.9$, $\beta_2 = 0.95$. We use a cosine learning rate schedule, such that the final learning rate is equal to 10% of the maximal learning rate. We use a weight decay of 0.1 and gradient clipping of 1.0. We use 2,000 warmup

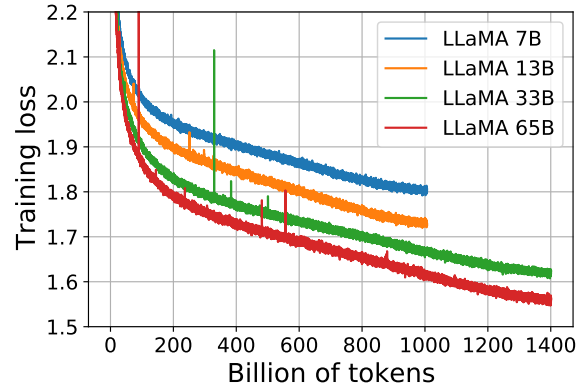


Figure 1: Training loss over train tokens for the 7B, 13B, 33B, and 65 models. LLaMA-33B and LLaMA-65B were trained on 1.4T tokens. The smaller models were trained on 1.0T tokens. All models are trained with a batch size of 4M tokens.

steps, and vary the learning rate and batch size with the size of the model (see Table 2 for details).

2.4 Efficient implementation

We make several optimizations to improve the training speed of our models. First, we use an efficient implementation of the causal multi-head attention to reduce memory usage and runtime. This implementation, available in the xformers library,² is inspired by Rabe and Staats (2021) and uses the backward from Dao et al. (2022). This is achieved by not storing the attention weights and not computing the key/query scores that are masked due to the causal nature of the language modeling task.

To further improve training efficiency, we reduced the amount of activations that are recomputed during the backward pass with checkpointing. More precisely, we save the activations that are expensive to compute, such as the outputs of linear layers. This is achieved by manually implementing the backward function for the transformer layers, instead of relying on the PyTorch autograd. To fully benefit from this optimization, we need to

²<https://github.com/facebookresearch/xformers>

The authors of He et al. [2020] argued that the relative positions of two tokens could only be fully modeled using the middle two terms of Equation (6). As a consequence, the absolute position embeddings $\mathbf{p}_m$ and $\mathbf{p}_n$ were simply replaced with the relative position embeddings $\tilde{\mathbf{p}}_{m-n}$:

$$\mathbf{q}_m^\top \mathbf{k}_n = \mathbf{x}_m^\top \mathbf{W}_q^\top \mathbf{W}_k \mathbf{x}_n + \mathbf{x}_m^\top \mathbf{W}_q^\top \mathbf{W}_k \tilde{\mathbf{p}}_{m-n} + \tilde{\mathbf{p}}_{m-n}^\top \mathbf{W}_q^\top \mathbf{W}_k \mathbf{x}_n \quad (10)$$

A comparison of the four variants of the relative position embeddings Radford and Narasimhan [2018] has shown that the variant similar to Equation (10) is the most efficient among the other three. Generally speaking, all these approaches attempt to modify Equation (6) based on the decomposition of Equation (3) under the self-attention settings in Equation (2), which was originally proposed in Vaswani et al. [2017]. They commonly introduced to directly add the position information to the context representations. Unlikely, our approach aims to derive the relative position encoding from Equation (1) under some constraints. Next, we show that the derived approach is more interpretable by incorporating relative position information with the rotation of context representations.

3 Proposed approach

In this section, we discuss the proposed rotary position embedding (RoPE). We first formulate the relative position encoding problem in Section (3.1), we then derive the RoPE in Section (3.2) and investigate its properties in Section (3.3).

3.1 Formulation

Transformer-based language modeling usually leverages the position information of individual tokens through a self-attention mechanism. As can be observed in Equation (2), $\mathbf{q}_m^\top \mathbf{k}_n$ typically enables knowledge conveyance between tokens at different positions. In order to incorporate relative position information, we require the inner product of query $\mathbf{q}_m$ and key $\mathbf{k}_n$ to be formulated by a function g , which takes only the word embeddings $\mathbf{x}_m, \mathbf{x}_n$, and their relative position $m - n$ as input variables. In other words, we hope that the inner product encodes position information only in the relative form:

$$\langle f_q(\mathbf{x}_m, m), f_k(\mathbf{x}_n, n) \rangle = g(\mathbf{x}_m, \mathbf{x}_n, m - n). \quad (11)$$

The ultimate goal is to find an equivalent encoding mechanism to solve the functions $f_q(\mathbf{x}_m, m)$ and $f_k(\mathbf{x}_n, n)$ to conform the aforementioned relation.

3.2 Rotary position embedding

3.2.1 A 2D case

We begin with a simple case with a dimension $d = 2$. Under these settings, we make use of the geometric property of vectors on a 2D plane and its complex form to prove (refer Section (3.4.1) for more details) that a solution to our formulation Equation (11) is:

$$\begin{aligned} f_q(\mathbf{x}_m, m) &= (\mathbf{W}_q \mathbf{x}_m) e^{im\theta} \\ f_k(\mathbf{x}_n, n) &= (\mathbf{W}_k \mathbf{x}_n) e^{in\theta} \\ g(\mathbf{x}_m, \mathbf{x}_n, m - n) &= \text{Re}[(\mathbf{W}_q \mathbf{x}_m)(\mathbf{W}_k \mathbf{x}_n)^* e^{i(m-n)\theta}] \end{aligned} \quad (12)$$

where $\text{Re}[\cdot]$ is the real part of a complex number and $(\mathbf{W}_k \mathbf{x}_n)^*$ represents the conjugate complex number of $(\mathbf{W}_k \mathbf{x}_n)$. $\theta \in \mathbb{R}$ is a preset non-zero constant. We can further write $f_{\{q,k\}}$ in a multiplication matrix:

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} W_{\{q,k\}}^{(11)} & W_{\{q,k\}}^{(12)} \\ W_{\{q,k\}}^{(21)} & W_{\{q,k\}}^{(22)} \end{pmatrix} \begin{pmatrix} x_m^{(1)} \\ x_m^{(2)} \end{pmatrix} \quad (13)$$

where $(x_m^{(1)}, x_m^{(2)})$ is $\mathbf{x}_m$ expressed in the 2D coordinates. Similarly, g can be viewed as a matrix and thus enables the solution of formulation in Section (3.1) under the 2D case. Specifically, incorporating the relative position embedding is straightforward: simply rotate the affine-transformed word embedding vector by amount of angle multiples of its position index and thus interprets the intuition behind *Rotary Position Embedding*.

3.2.2 General form

In order to generalize our results in 2D to any $\mathbf{x}_i \in \mathbb{R}^d$ where d is even, we divide the d -dimension space into $d/2$ sub-spaces and combine them in the merit of the linearity of the inner product, turning $f_{\{q,k\}}$ into:

$$f_{\{q,k\}}(\mathbf{x}_m, m) = \mathbf{R}_{\Theta, m}^d \mathbf{W}_{\{q,k\}} \mathbf{x}_m \quad (14)$$

where

$$\mathbf{R}_{\Theta, m}^d = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix} \quad (15)$$

is the rotary matrix with pre-defined parameters $\Theta = \{\theta_i = 10000^{-2(i-1)/d}, i \in [1, 2, \dots, d/2]\}$. A graphic illustration of RoPE is shown in Figure (1). Applying our RoPE to self-attention in Equation (2), we obtain:

$$\mathbf{q}_m^\top \mathbf{k}_n = (\mathbf{R}_{\Theta, m}^d \mathbf{W}_q \mathbf{x}_m)^\top (\mathbf{R}_{\Theta, n}^d \mathbf{W}_k \mathbf{x}_n) = \mathbf{x}^\top \mathbf{W}_q \mathbf{R}_{\Theta, n-m}^d \mathbf{W}_k \mathbf{x}_n \quad (16)$$

where $\mathbf{R}_{\Theta, n-m}^d = (\mathbf{R}_{\Theta, m}^d)^\top \mathbf{R}_{\Theta, n}^d$. Note that $\mathbf{R}_{\Theta}^d$ is an orthogonal matrix, which ensures stability during the process of encoding position information. In addition, due to the sparsity of $\mathbf{R}_{\Theta}^d$, applying matrix multiplication directly as in Equation (16) is not computationally efficient; we provide another realization in theoretical explanation.

In contrast to the additive nature of position embedding method adopted in the previous works, i.e., Equations (3) to (10), our approach is multiplicative. Moreover, RoPE naturally incorporates relative position information through rotation matrix product instead of altering terms in the expanded formulation of additive position encoding when applied with self-attention.

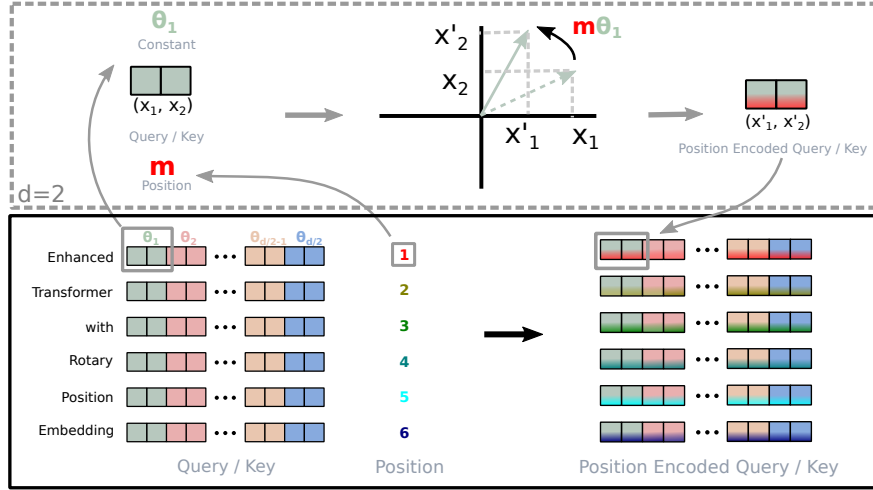


Figure 1: Implementation of Rotary Position Embedding (RoPE).

3.3 Properties of RoPE

Long-term decay: Following Vaswani et al. [2017], we set $\theta_i = 10000^{-2i/d}$. One can prove that this setting provides a long-term decay property (refer to Section (3.4.3) for more details), which means the inner-product will decay when the relative position increase. This property coincides with the intuition that a pair of tokens with a long relative distance should have less connection.

RoPE with linear attention: The self-attention can be rewritten in a more general form.

RoFormer

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})_m = \frac{\sum_{n=1}^N \text{sim}(\mathbf{q}_m, \mathbf{k}_n) \mathbf{v}_n}{\sum_{n=1}^N \text{sim}(\mathbf{q}_m, \mathbf{k}_n)}. \quad (17)$$

The original self-attention chooses $\text{sim}(\mathbf{q}_m, \mathbf{k}_n) = \exp(\mathbf{q}_m^\top \mathbf{k}_n / \sqrt{d})$. Note that the original self-attention should compute the inner product of query and key for every pair of tokens, which has a quadratic complexity $\mathcal{O}(N^2)$. Follow Katharopoulos et al. [2020], the linear attentions reformulate Equation (17) as

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})_m = \frac{\sum_{n=1}^N \phi(\mathbf{q}_m)^\top \varphi(\mathbf{k}_n) \mathbf{v}_n}{\sum_{n=1}^N \phi(\mathbf{q}_m)^\top \varphi(\mathbf{k}_n)}, \quad (18)$$

where $\phi(\cdot), \varphi(\cdot)$ are usually non-negative functions. The authors of Katharopoulos et al. [2020] have proposed $\phi(x) = \varphi(x) = \text{elu}(x) + 1$ and first computed the multiplication between keys and values using the associative property of matrix multiplication. A softmax function is used in Shen et al. [2021] to normalize queries and keys separately before the inner product, which is equivalent to $\phi(\mathbf{q}_i) = \text{softmax}(\mathbf{q}_i)$ and $\varphi(\mathbf{k}_j) = \exp(\mathbf{k}_j)$. For more details about linear attention, we encourage readers to refer to original papers. In this section, we focus on discussing incorporating RoPE with Equation (18). Since RoPE injects position information by rotation, which keeps the norm of hidden representations unchanged, we can combine RoPE with linear attention by multiplying the rotation matrix with the outputs of the non-negative functions.

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})_m = \frac{\sum_{n=1}^N (\mathbf{R}_{\Theta, m}^d \phi(\mathbf{q}_m))^\top (\mathbf{R}_{\Theta, n}^d \varphi(\mathbf{k}_n)) \mathbf{v}_n}{\sum_{n=1}^N \phi(\mathbf{q}_m)^\top \varphi(\mathbf{k}_n)}. \quad (19)$$

It is noteworthy that we keep the denominator unchanged to avoid the risk of dividing zero, and the summation in the numerator could contain negative terms. Although the weights for each value $\mathbf{v}_i$ in Equation (19) are not strictly probabilistic normalized, we kindly argue that the computation can still model the importance of values.

3.4 Theoretical Explanation

3.4.1 Derivation of RoPE under 2D

Under the case of $d = 2$, we consider two-word embedding vectors $\mathbf{x}_q, \mathbf{x}_k$ corresponds to query and key and their position m and n , respectively. According to eq. (1), their position-encoded counterparts are:

$$\begin{aligned} \mathbf{q}_m &= f_q(\mathbf{x}_q, m), \\ \mathbf{k}_n &= f_k(\mathbf{x}_k, n), \end{aligned} \quad (20)$$

where the subscripts of $\mathbf{q}_m$ and $\mathbf{k}_n$ indicate the encoded positions information. Assume that there exists a function g that defines the inner product between vectors produced by $f_{\{q, k\}}$:

$$\mathbf{q}_m^\top \mathbf{k}_n = \langle f_q(\mathbf{x}_q, m), f_k(\mathbf{x}_k, n) \rangle = g(\mathbf{x}_q, \mathbf{x}_k, n - m), \quad (21)$$

we further require below initial condition to be satisfied:

$$\begin{aligned} \mathbf{q} &= f_q(\mathbf{x}_q, 0), \\ \mathbf{k} &= f_k(\mathbf{x}_k, 0), \end{aligned} \quad (22)$$

which can be read as the vectors with empty position information encoded. Given these settings, we attempt to find a solution of f_q, f_k . First, we take advantage of the geometric meaning of vector in 2D and its complex counter part, decompose functions in Equations (20) and (21) into:

$$\begin{aligned} f_q(\mathbf{x}_q, m) &= R_q(\mathbf{x}_q, m) e^{i\Theta_q(\mathbf{x}_q, m)}, \\ f_k(\mathbf{x}_k, n) &= R_k(\mathbf{x}_k, n) e^{i\Theta_k(\mathbf{x}_k, n)}, \\ g(\mathbf{x}_q, \mathbf{x}_k, n - m) &= R_g(\mathbf{x}_q, \mathbf{x}_k, n - m) e^{i\Theta_g(\mathbf{x}_q, \mathbf{x}_k, n - m)}, \end{aligned} \quad (23)$$

where R_f, R_g and Θ_f, Θ_g are the radical and angular components for $f_{\{q, k\}}$ and g , respectively. Plug them into Equation (21), we get the relation:

$$\begin{aligned} R_q(\mathbf{x}_q, m) R_k(\mathbf{x}_k, n) &= R_g(\mathbf{x}_q, \mathbf{x}_k, n - m), \\ \Theta_k(\mathbf{x}_k, n) - \Theta_q(\mathbf{x}_q, m) &= \Theta_g(\mathbf{x}_q, \mathbf{x}_k, n - m), \end{aligned} \quad (24)$$

RoFormer

with the corresponding initial condition as:

$$\begin{aligned}\mathbf{q} &= \|\mathbf{q}\|e^{i\theta_q} = R_q(\mathbf{x}_q, 0)e^{i\Theta_q(\mathbf{x}_q, 0)}, \\ \mathbf{k} &= \|\mathbf{k}\|e^{i\theta_k} = R_k(\mathbf{x}_k, 0)e^{i\Theta_k(\mathbf{x}_k, 0)},\end{aligned}\tag{25}$$

where $\|\mathbf{q}\|$, $\|\mathbf{k}\|$ and θ_q , θ_k are the radial and angular part of $\mathbf{q}$ and $\mathbf{k}$ on the 2D plane.

Next, we set $m = n$ in Equation (24) and take into account initial conditions in Equation (25):

$$R_q(\mathbf{x}_q, m)R_k(\mathbf{x}_k, m) = R_g(\mathbf{x}_q, \mathbf{x}_k, 0) = R_q(\mathbf{x}_q, 0)R_k(\mathbf{x}_k, 0) = \|\mathbf{q}\|\|\mathbf{k}\|,\tag{26a}$$

$$\Theta_k(\mathbf{x}_k, m) - \Theta_q(\mathbf{x}_q, m) = \Theta_g(\mathbf{x}_q, \mathbf{x}_k, 0) = \Theta_k(\mathbf{x}_k, 0) - \Theta_q(\mathbf{x}_q, 0) = \theta_k - \theta_q.\tag{26b}$$

On one hand, from, a straightforward solution of R_f could be formed from Equation (26a) :

$$\begin{aligned}R_q(\mathbf{x}_q, m) &= R_q(\mathbf{x}_q, 0) = \|\mathbf{q}\| \\ R_k(\mathbf{x}_k, n) &= R_k(\mathbf{x}_k, 0) = \|\mathbf{k}\| \\ R_g(\mathbf{x}_q, \mathbf{x}_k, n - m) &= R_g(\mathbf{x}_q, \mathbf{x}_k, 0) = \|\mathbf{q}\|\|\mathbf{k}\|\end{aligned}\tag{27}$$

which interprets the radial functions R_q , R_k and R_g are independent from the position information. On the other hand, as can be noticed in Equation (26b), $\Theta_q(\mathbf{x}_q, m) - \theta_q = \Theta_k(\mathbf{x}_k, m) - \theta_k$ indicates that the angular functions does not dependent on query and key, we set them to $\Theta_f := \Theta_q = \Theta_k$ and term $\Theta_f(\mathbf{x}_{\{q,k\}}, m) - \theta_{\{q,k\}}$ is a function of position m and is independent of word embedding $\mathbf{x}_{\{q,k\}}$, we denote it as $\phi(m)$, yielding:

$$\Theta_f(\mathbf{x}_{\{q,k\}}, m) = \phi(m) + \theta_{\{q,k\}},\tag{28}$$

Further, by plugging $n = m + 1$ to Equation (24) and consider the above equation, we can get:

$$\phi(m + 1) - \phi(m) = \Theta_g(\mathbf{x}_q, \mathbf{x}_k, 1) + \theta_q - \theta_k,\tag{29}$$

Since RHS is a constant irrelevant to m , $\phi(m)$ with continuous integer inputs produce an arithmetic progression:

$$\phi(m) = m\theta + \gamma,\tag{30}$$

where $\theta, \gamma \in \mathbb{R}$ are constants and θ is non-zero. To summarize our solutions from Equations (27) to (30):

$$\begin{aligned}f_q(\mathbf{x}_q, m) &= \|\mathbf{q}\|e^{i\theta_q + m\theta + \gamma} = \mathbf{q}e^{i(m\theta + \gamma)}, \\ f_k(\mathbf{x}_k, n) &= \|\mathbf{k}\|e^{i\theta_k + n\theta + \gamma} = \mathbf{k}e^{i(n\theta + \gamma)}.\end{aligned}\tag{31}$$

Note that we do not apply any constrains to f_q and f_k of Equation (22), thus $f_q(\mathbf{x}_m, 0)$ and $f_k(\mathbf{x}_n, 0)$ are left to choose freely. To make our results comparable to Equation (3), we define:

$$\begin{aligned}\mathbf{q} &= f_q(\mathbf{x}_m, 0) = \mathbf{W}_q \mathbf{x}_m, \\ \mathbf{k} &= f_k(\mathbf{x}_n, 0) = \mathbf{W}_k \mathbf{x}_n.\end{aligned}\tag{32}$$

Then, we simply set $\gamma = 0$ in Equation (31) of the final solution:

$$\begin{aligned}f_q(\mathbf{x}_m, m) &= (\mathbf{W}_q \mathbf{x}_m)e^{im\theta}, \\ f_k(\mathbf{x}_n, n) &= (\mathbf{W}_k \mathbf{x}_n)e^{in\theta}.\end{aligned}\tag{33}$$

3.4.2 Computational efficient realization of rotary matrix multiplication

Taking the advantage of the sparsity of $\mathbf{R}_{\Theta, m}^d$ in Equation (15), a more computational efficient realization of a multiplication of $\mathbf{R}_{\Theta}^d$ and $\mathbf{x} \in \mathbb{R}^d$ is:

$$\mathbf{R}_{\Theta, m}^d \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_{d-1} \\ x_d \end{pmatrix} \otimes \begin{pmatrix} \cos m\theta_1 \\ \cos m\theta_1 \\ \cos m\theta_2 \\ \cos m\theta_2 \\ \vdots \\ \cos m\theta_{d/2} \\ \cos m\theta_{d/2} \end{pmatrix} + \begin{pmatrix} -x_2 \\ x_1 \\ -x_4 \\ x_3 \\ \vdots \\ -x_d \\ x_{d-1} \end{pmatrix} \otimes \begin{pmatrix} \sin m\theta_1 \\ \sin m\theta_1 \\ \sin m\theta_2 \\ \sin m\theta_2 \\ \vdots \\ \sin m\theta_{d/2} \\ \sin m\theta_{d/2} \end{pmatrix}\tag{34}$$

Different from these related work, the proposed RMSNorm modifies the normalization structure by removing the re-centering operation and regularizing the summed inputs with RMS alone. Our model only maintains the re-scaling invariance property which we find can be inherited when the RMS is estimated from only subset of the summed inputs, partially inspired by the group normalization [34]. As a side effect, our model reduces the computational overhead and increases efficiency. Recently, Zhang et al. [36] show that with careful initialization, residual networks can be trained as stable as those with normalization. However, the approach mainly aims at improving residual networks and can not be freely switched without modifying all initialization layers. Besides, it is not trivial to be adapted to other general neural networks, such as RNNs where model depth expands along the variable sequence length. By contrast, our model is simple, effective and can be used as a drop-in replacement of LayerNorm.

3 Background

We briefly review LayerNorm in this section based on a standard feed-forward neural network. Given an input vector $\mathbf{x} \in \mathbb{R}^m$, a feed-forward network projects it into an output vector $\mathbf{y} \in \mathbb{R}^n$ through a linear transformation followed by a non-linear activation as follows:

$$a_i = \sum_{j=1}^m w_{ij}x_j, \quad y_i = f(a_i + b_i), \quad (1)$$

where w_i is weight vector to the i -th output neuron, b_i is bias scalar which is usually initialized by 0, and $f(\cdot)$ is an element-wise non-linear function. $\mathbf{a} \in \mathbb{R}^n$ denotes the weight-summed inputs to neurons, which is also the target of normalization.

This vanilla network might suffer from *internal covariate shift* issue [12], where a layer's input distribution changes as previous layers are updated. This could negatively affect the stability of parameters' gradients, delaying model convergence. To reduce this shift, LayerNorm normalizes the summed inputs so as to fix their mean and variance as follows:

$$\bar{a}_i = \frac{a_i - \mu}{\sigma} g_i, \quad y_i = f(\bar{a}_i + b_i), \quad (2)$$

where $\bar{a}_i$ is the i -th value of vector $\bar{\mathbf{a}} \in \mathbb{R}^n$, which acts as the normalized alternative of a_i for layer activation. $\mathbf{g} \in \mathbb{R}^n$ is the gain parameter used to re-scale the standardized summed inputs, and is set to 1 at the beginning. μ and σ^2 are the mean and variance statistic respectively estimated from raw summed inputs $\mathbf{a}$:

$$\mu = \frac{1}{n} \sum_{i=1}^n a_i, \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (a_i - \mu)^2}. \quad (3)$$

Thus, LayerNorm forces the norm of neurons to be decoupled from the inputs and weight matrix.

4 RMSNorm

A well-known explanation of the success of LayerNorm is its re-centering and re-scaling invariance property. The former enables the model to be insensitive to shift noises on both inputs and weights, and the latter keeps the output representations intact when both inputs and weights are randomly scaled. In this paper, we hypothesize that the re-scaling invariance is the reason for success of LayerNorm, rather than re-centering invariance.

We propose RMSNorm which only focuses on re-scaling invariance and regularizes the summed inputs simply according to the root mean square (RMS) statistic:

$$\bar{a}_i = \frac{a_i}{\text{RMS}(\mathbf{a})} g_i, \quad \text{where } \text{RMS}(\mathbf{a}) = \sqrt{\frac{1}{n} \sum_{i=1}^n a_i^2}. \quad (4)$$

Intuitively, RMSNorm simplifies LayerNorm by totally removing the mean statistic in Eq. (3) at the cost of sacrificing the invariance that mean normalization affords. When the mean of summed inputs is zero, RMSNorm is exactly equal to LayerNorm. Although RMSNorm does not re-center

	Weight matrix re-scaling	Weight matrix re-centering	Weight vector re-scaling	Dataset re-scaling	Dataset re-centering	Single training case re-scaling
BatchNorm	✓	✗	✓	✓	✓	✗
WeightNorm	✓	✗	✓	✗	✗	✗
LayerNorm	✓	✓	✗	✓	✗	✓
RMSNorm	✓	✗	✗	✓	✗	✓
pRMSNorm	✓	✗	✗	✓	✗	✓

Table 1: Invariance properties of different normalization methods. “✓” indicates invariant, while “✗” denotes the opposite.

the summed inputs as in LayerNorm, we demonstrate through experiments that this property is not fundamental to the success of LayerNorm, and that RMSNorm is similarly or more effective.

RMS measures the quadratic mean of inputs, which in RMSNorm forces the summed inputs into a $\sqrt{n}$ -scaled unit sphere. By doing so, the output distribution remains regardless of the scaling of input and weight distributions, benefiting the stability of layer activations. Although Euclidean norm which only differs from RMS by a factor of $\sqrt{n}$ has been successfully explored [22], we empirically find that it does not work for layer normalization. We hypothesize that scaling the sphere with the size of the input vector is important because it makes the normalization more robust across vectors of different size. As far as we know, the idea of employing RMS for neural network normalization has not been investigated before.

4.1 Invariance Analysis

Invariance measures whether model output after normalization changes highly in accordance with its input and weight matrix. Ba et al. [3] show that different normalization methods reveal different invariance properties, which contributes considerably to the model’s robustness. In this section, we theoretically examine the invariance properties of RMSNorm.

We consider the following general form of RMSNorm:

$$\mathbf{y} = f\left(\frac{\mathbf{W}\mathbf{x}}{\text{RMS}(\mathbf{a})} \odot \mathbf{g} + \mathbf{b}\right), \quad (5)$$

where $\odot$ denotes element-wise multiplication. Our main results are summarized in Table 1. RMSNorm is invariant to both weight matrix and input re-scaling, because of the following linearity property of RMS:

$$\text{RMS}(\alpha\mathbf{x}) = \alpha\text{RMS}(\mathbf{x}), \quad (6)$$

where α is a scale value. Suppose the weight matrix is scaled by a factor of δ , i.e. $\mathbf{W}' = \delta\mathbf{W}$, then this change does not affect the final layer output:

$$\mathbf{y}' = f\left(\frac{\mathbf{W}'\mathbf{x}}{\text{RMS}(\mathbf{a}')} \odot \mathbf{g} + \mathbf{b}\right) = f\left(\frac{\delta\mathbf{W}\mathbf{x}}{\delta\text{RMS}(\mathbf{a})} \odot \mathbf{g} + \mathbf{b}\right) = \mathbf{y}. \quad (7)$$

By contrast, if the scaling is only performed on individual weight vectors, this property does not hold anymore as different scaling factors break the linearity property of RMS. Similarly, if we enforce a scale on the input with a factor of δ , i.e. $\mathbf{x}' = \delta\mathbf{x}$, the output of RMSNorm remains through an analysis analogous to that in Eq. 7. We can easily extend the equality to batch-based inputs as well as the whole dataset. Therefore, RMSNorm is invariant to the scaling of its inputs.

The main difference to LayerNorm is that RMSNorm is not re-centered and thus does not show similar linearity property for variable shifting. It is not invariant to all re-centering operations.

4.2 Gradient Analysis

The above analysis only considers the effect of scaling inputs and the weight matrix on the layer output. In a general setting, however, a RMSNorm-enhanced neural network is trained via standard stochastic gradient descent approach, where the robustness of model gradient is very crucial to parameters’ update and model convergence (see also Santurkar et al. [23] who argue that the success of normalization methods does not come from the added stability to layer inputs, but due to increased smoothness of the optimization landscape). In this section, we investigate the properties of model gradients in RMSNorm.

Given a loss function $\mathcal{L}$, we perform back-propagation through Eq. (4) to obtain the gradient with respect to parameters $\mathbf{g}$, $\mathbf{b}$ as follows:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \frac{\partial \mathcal{L}}{\partial \mathbf{v}}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{g}} = \frac{\partial \mathcal{L}}{\partial \mathbf{v}} \odot \frac{\mathbf{W}\mathbf{x}}{\text{RMS}(\mathbf{a})}, \quad (8)$$

where $\mathbf{v}$ is short for the whole expression inside $f(\cdot)$ in Eq. (4), and $\partial \mathcal{L} / \partial \mathbf{v}$ is the gradient back-propagated from $\mathcal{L}$ to $\mathbf{v}$. Both gradients $\partial \mathcal{L} / \partial \mathbf{b}$ and $\partial \mathcal{L} / \partial \mathbf{g}$ are invariant to the scaling of inputs $\mathbf{x}$ and the weight matrix $\mathbf{W}$ (in the case of $\partial \mathcal{L} / \partial \mathbf{g}$ because of the linearity property in Eq. (6)). Besides, the gradient of $\mathbf{g}$ is proportional to the normalized summed inputs, rather than raw inputs. This powers the stability of the magnitude of $\mathbf{g}$.

Unlike these vector parameters, the gradient of the weight matrix $\mathbf{W}$ is more complicated due to the quadratic computation in RMS. Formally,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \sum_{i=1}^n \left[\mathbf{x}^T \otimes \left(\text{diag} \left(\mathbf{g} \odot \frac{\partial \mathcal{L}}{\partial \mathbf{v}} \right) \times \mathbf{R} \right) \right]_i, \text{ where } \mathbf{R} = \frac{1}{\text{RMS}(\mathbf{a})} \left(\mathbf{I} - \frac{(\mathbf{W}\mathbf{x})(\mathbf{W}\mathbf{x})^T}{n\text{RMS}(\mathbf{a})^2} \right), \quad (9)$$

$\text{diag}(\cdot)$ denotes the diagonal matrix of input, $\otimes$ denotes the Kronecker product, and “ $\mathbf{I}$ ” indicates identity matrix. For clarity, we explicitly use “ $\times$ ” to represent matrix multiplication. The matrix term $\mathbf{R}$ associates the gradient of $\mathbf{W}$ with both inputs $\mathbf{x}$ and weight matrix $\mathbf{W}$. With a thorough analysis, we can demonstrate that this term is negatively correlated with both input and weight matrix scaling. After assigning a scale of δ to either input $\mathbf{x}$ ($\mathbf{x}' = \delta \mathbf{x}$) or weight matrix ($\mathbf{W}' = \delta \mathbf{W}$), we have

$$\mathbf{R}' = \frac{1}{\delta \text{RMS}(\mathbf{a})} \left(\mathbf{I} - \frac{(\delta \mathbf{W}\mathbf{x})(\delta \mathbf{W}\mathbf{x})^T}{n\delta^2 \text{RMS}(\mathbf{a})^2} \right) = \frac{1}{\delta} \mathbf{R}. \quad (10)$$

If we put the scaled term $\mathbf{R}'$ back into Eq. (9), we can easily prove that the gradient $\partial \mathcal{L} / \partial \mathbf{W}$ is invariant to input scaling, but keeps the negative correlation with weight matrix scaling. Reducing the sensitivity of gradient $\partial \mathcal{L} / \partial \mathbf{W}$ to the scaling of inputs ensures its smoothness and improves the stability of learning. On the other hand, the negative correlation acts as an implicit learning rate adaptor and dynamically controls the norm of gradients which avoids large-norm weight matrix and improves model convergence.

5 p RMSNorm

The re-scaling invariance property of RMSNorm ascribes to the linearity property of RMS. Considering that neurons in one layer often have independent identically distributed structure, we argue that the RMS can be estimated on a subset of these neurons rather than all of them. We propose partial RMSNorm (p RMSNorm). Given the unnormalized input $\mathbf{a}$, p RMSNorm infers the RMS statistic from first- $p\%$ elements of $\mathbf{a}$: $\overline{\text{RMS}}(\mathbf{a}) = \sqrt{\frac{1}{k} \sum_{i=1}^k a_i^2}$, where $k = \lceil n \cdot p \rceil$ denotes the number of elements used for RMS estimation. The linearity property still holds for $\overline{\text{RMS}}$ as in Eq. (6), which indicates p RMSNorm shares the same invariance properties as RMSNorm as shown in Table 1.

$\overline{\text{RMS}}$ is a biased estimation of the RMS which is often inaccurate. Though theoretically p RMSNorm approximates to RMSNorm, we observe gradient instability where the gradient tends to explode with small m . In practice, however, models with p RMSNorm can succeed in satisfactory convergence with a partial ratio of 6.25%.

6 Experiments

To test the efficiency of layer normalization across different implementations, we perform experiments with Tensorflow [1], PyTorch [20] and Theano [29]. We add RMSNorm to different models, comparing against an unnormalized baseline and LayerNorm. These models are based on diverse architectures, covering different RNN variants, convolutional and self-attentional models, and various activations (such as sigmoid, tanh, and softmax), with initialization ranging from uniform, normal, orthogonal with different initialization ranges or variances. Unless otherwise noted, all speed-related statistics are measured on one TITAN X (Pascal). Reported time is averaged over 3 runs. We also list the standard deviation of these three runs.

GLU Variants Improve Transformer

Noam Shazeer
Google
noam@google.com

February 14, 2020

Abstract

Gated Linear Units [Dauphin et al., 2016] consist of the component-wise product of two linear projections, one of which is first passed through a sigmoid function. Variations on GLU are possible, using different nonlinear (or even linear) functions in place of sigmoid. We test these variants in the feed-forward sublayers of the Transformer [Vaswani et al., 2017] sequence-to-sequence model, and find that some of them yield quality improvements over the typically-used ReLU or GELU activations.

1 Introduction

The Transformer [Vaswani et al., 2017] sequence-to-sequence model alternates between multi-head attention, and what it calls "position-wise feed-forward networks" (FFN). The FFN takes a vector x (the hidden representation at a particular position in the sequence) and passes it through two learned linear transformations, (represented by the matrices W_1 and W_2 and bias vectors b_1 and b_2). A rectified-linear (ReLU) [Glorot et al., 2011] activation function applied between the two linear transformations.

$$\text{FFN}(x, W_1, W_2, b_1, b_2) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (1)$$

Following the T5 codebase [Raffel et al., 2019]¹, we use a version with no bias:

$$\text{FFN}_{\text{ReLU}}(x, W_1, W_2) = \max(xW_1, 0)W_2 \quad (2)$$

Subsequent work has proposed replacing the ReLU with other nonlinear activation functions such as Gaussian Error Linear Units, $\text{GELU}(x) = x\Phi(x)$ [Hendrycks and Gimpel, 2016], and Swish $_{\beta}(x) = x\sigma(\beta x)$ [Ramachandran et al., 2017].

$$\begin{aligned} \text{FFN}_{\text{GELU}}(x, W_1, W_2) &= \text{GELU}(xW_1)W_2 \\ \text{FFN}_{\text{Swish}}(x, W_1, W_2) &= \text{Swish}_1(xW_1)W_2 \end{aligned} \quad (3)$$

2 Gated Linear Units (GLU) and Variants

[Dauphin et al., 2016] introduced Gated Linear Units (GLU), a neural network layer defined as the component-wise product of two linear transformations of the input, one of which is sigmoid-activated. They also suggest omitting the activation, which they call a "bilinear" layer and attribute to [Mnih and Hinton, 2007].

$$\begin{aligned} \text{GLU}(x, W, V, b, c) &= \sigma(xW + b) \otimes (xV + c) \\ \text{Bilinear}(x, W, V, b, c) &= (xW + b) \otimes (xV + c) \end{aligned} \quad (4)$$

We can also define GLU variants using other activation functions:

¹Also in the interest of ML fairness.

$$\begin{aligned}
\text{ReGLU}(x, W, V, b, c) &= \max(0, xW + b) \otimes (xV + c) \\
\text{GEGLU}(x, W, V, b, c) &= \text{GELU}(xW + b) \otimes (xV + c) \\
\text{SwiGLU}(x, W, V, b, c, \beta) &= \text{Swish}_\beta(xW + b) \otimes (xV + c)
\end{aligned} \tag{5}$$

In this paper, we propose additional variations on the Transformer FFN layer which use GLU or one of its variants in place of the first linear transformation and the activation function. Again, we omit the bias terms.

$$\begin{aligned}
\text{FFN}_{\text{GLU}}(x, W, V, W_2) &= (\sigma(xW) \otimes xV)W_2 \\
\text{FFN}_{\text{Bilinear}}(x, W, V, W_2) &= (xW \otimes xV)W_2 \\
\text{FFN}_{\text{ReGLU}}(x, W, V, W_2) &= (\max(0, xW) \otimes xV)W_2 \\
\text{FFN}_{\text{GEGLU}}(x, W, V, W_2) &= (\text{GELU}(xW) \otimes xV)W_2 \\
\text{FFN}_{\text{SwiGLU}}(x, W, V, W_2) &= (\text{Swish}_1(xW) \otimes xV)W_2
\end{aligned} \tag{6}$$

All of these layers have three weight matrices, as opposed to two for the original FFN. To keep the number of parameters and the amount of computation constant, we reduce the number of hidden units d_{ff} (the second dimension of W and V and the first dimension of W_2) by a factor of $\frac{2}{3}$ when comparing these layers to the original two-matrix version.

3 Experiments on Text-to-Text Transfer Transformer (T5)

We test the FFN variants we have described on the transfer-learning setup from [Raffel et al., 2019]. An encoder-decoder transformer model [Vaswani et al., 2017] is trained on a denoising objective of predicting missing text segments, and subsequently fine-tuned on various language understanding tasks.

3.1 Model Architecture

We use the same code base, model architecture, and training task as the base model from [Raffel et al., 2019]. The encoder and decoder each consist of 12 layers, with $d_{model} = 768$. For the attention layers, $h = 12$ and $d_k = d_v = 64$. The FFN layers have hidden size $d_{ff} = 3072$. As we describe above, for the GLU-variant-based FFN layers, which have three weight matrices instead of two, we reduce the hidden layer to $d_{ff} = 2048$, so as to maintain the same parameter and operation counts as the base model.

Table 1: Heldout-set log-perplexity for Transformer models on the segment-filling task from [Raffel et al., 2019]. All models are matched for parameters and computation.

Training Steps	65,536	524,288
$\text{FFN}_{\text{ReLU}}(\text{baseline})$	1.997 (0.005)	1.677
FFN_{GELU}	1.983 (0.005)	1.679
$\text{FFN}_{\text{Swish}}$	1.994 (0.003)	1.683
FFN_{GLU}	1.982 (0.006)	1.663
$\text{FFN}_{\text{Bilinear}}$	1.960 (0.005)	1.648
$\text{FFN}_{\text{GEGLU}}$	1.942 (0.004)	1.633
$\text{FFN}_{\text{SwiGLU}}$	1.944 (0.010)	1.636
$\text{FFN}_{\text{ReGLU}}$	1.953 (0.003)	1.645

pothesis in Sections 4 and 5. First, we discuss different subword representations.

3.1 Related Work

For Statistical Machine Translation (SMT), the translation of unknown words has been the subject of intensive research.

A large proportion of unknown words are names, which can just be copied into the target text if both languages share an alphabet. If alphabets differ, transliteration is required (Durrani et al., 2014). Character-based translation has also been investigated with phrase-based models, which proved especially successful for closely related languages (Vilar et al., 2007; Tiedemann, 2009; Neubig et al., 2012).

The segmentation of morphologically complex words such as compounds is widely used for SMT, and various algorithms for morpheme segmentation have been investigated (Nießen and Ney, 2000; Koehn and Knight, 2003; Virpioja et al., 2007; Stallard et al., 2012). Segmentation algorithms commonly used for phrase-based SMT tend to be conservative in their splitting decisions, whereas we aim for an aggressive segmentation that allows for open-vocabulary translation with a compact network vocabulary, and without having to resort to back-off dictionaries.

The best choice of subword units may be task-specific. For speech recognition, phone-level language models have been used (Bazzi and Glass, 2000). Mikolov et al. (2012) investigate subword language models, and propose to use syllables. For multilingual segmentation tasks, multilingual algorithms have been proposed (Snyder and Barzilay, 2008). We find these intriguing, but inapplicable at test time.

Various techniques have been proposed to produce fixed-length continuous word vectors based on characters or morphemes (Luong et al., 2013; Botha and Blunsom, 2014; Ling et al., 2015a; Kim et al., 2015). An effort to apply such techniques to NMT, parallel to ours, has found no significant improvement over word-based approaches (Ling et al., 2015b). One technical difference from our work is that the attention mechanism still operates on the level of words in the model by Ling et al. (2015b), and that the representation of each word is fixed-length. We expect that the attention mechanism benefits from our variable-length representation: the network can learn to place atten-

tion on different subword units at each step. Recall our introductory example *Abwasserbehandlungsanlage*, for which a subword segmentation avoids the information bottleneck of a fixed-length representation.

Neural machine translation differs from phrase-based methods in that there are strong incentives to minimize the vocabulary size of neural models to increase time and space efficiency, and to allow for translation without back-off models. At the same time, we also want a compact representation of the text itself, since an increase in text length reduces efficiency and increases the distances over which neural models need to pass information.

A simple method to manipulate the trade-off between vocabulary size and text size is to use shortlists of unsegmented words, using subword units only for rare words. As an alternative, we propose a segmentation algorithm based on byte pair encoding (BPE), which lets us learn a vocabulary that provides a good compression rate of the text.

3.2 Byte Pair Encoding (BPE)

Byte Pair Encoding (BPE) (Gage, 1994) is a simple data compression technique that iteratively replaces the most frequent pair of bytes in a sequence with a single, unused byte. We adapt this algorithm for word segmentation. Instead of merging frequent pairs of bytes, we merge characters or character sequences.

Firstly, we initialize the symbol vocabulary with the character vocabulary, and represent each word as a sequence of characters, plus a special end-of-word symbol ‘.’, which allows us to restore the original tokenization after translation. We iteratively count all symbol pairs and replace each occurrence of the most frequent pair (‘A’, ‘B’) with a new symbol ‘AB’. Each merge operation produces a new symbol which represents a character n -gram. Frequent character n -grams (or whole words) are eventually merged into a single symbol, thus BPE requires no shortlist. The final symbol vocabulary size is equal to the size of the initial vocabulary, plus the number of merge operations – the latter is the only hyperparameter of the algorithm.

For efficiency, we do not consider pairs that cross word boundaries. The algorithm can thus be run on the dictionary extracted from a text, with each word being weighted by its frequency. A minimal Python implementation is shown in Al-

Algorithm 1 Learn BPE operations

```

import re, collections

def get_stats(vocab):
    pairs = collections.defaultdict(int)
    for word, freq in vocab.items():
        symbols = word.split()
        for i in range(len(symbols)-1):
            pairs[symbols[i], symbols[i+1]] += freq
    return pairs

def merge_vocab(pair, v_in):
    v_out = {}
    bigram = re.escape(' '.join(pair))
    p = re.compile(r'(?!\S)' + bigram + r'(?!\S)')
    for word in v_in:
        w_out = p.sub(' '.join(pair), word)
        v_out[w_out] = v_in[word]
    return v_out

vocab = {'l o w </w>' : 5, 'l o w e r </w>' : 2,
        'n e w e s t </w>' : 6, 'w i d e s t </w>' : 3}
num_merges = 10
for i in range(num_merges):
    pairs = get_stats(vocab)
    best = max(pairs, key=pairs.get)
    vocab = merge_vocab(best, vocab)
    print(best)

```

r ·	→	r·
l o	→	lo
l o w	→	low
e r ·	→	er·

Figure 1: BPE merge operations learned from dictionary {‘low’, ‘lowest’, ‘newer’, ‘wider’}.

gorithm 1. In practice, we increase efficiency by indexing all pairs, and updating data structures incrementally.

The main difference to other compression algorithms, such as Huffman encoding, which have been proposed to produce a variable-length encoding of words for NMT (Chitnis and DeNero, 2015), is that our symbol sequences are still interpretable as subword units, and that the network can generalize to translate and produce new words (unseen at training time) on the basis of these subword units.

Figure 1 shows a toy example of learned BPE operations. At test time, we first split words into sequences of characters, then apply the learned operations to merge the characters into larger, known symbols. This is applicable to any word, and allows for open-vocabulary networks with fixed symbol vocabularies.³ In our example, the OOV ‘lower’ would be segmented into ‘low er·’.

³The only symbols that will be unknown at test time are unknown characters, or symbols of which all occurrences in the training text have been merged into larger symbols, like ‘safeguar’, which has all occurrences in our training text merged into ‘safeguard’. We observed no such symbols at test time, but the issue could be easily solved by recursively reversing specific merges until all symbols are known.

We evaluate two methods of applying BPE: learning two independent encodings, one for the source, one for the target vocabulary, or learning the encoding on the union of the two vocabularies (which we call *joint BPE*).⁴ The former has the advantage of being more compact in terms of text and vocabulary size, and having stronger guarantees that each subword unit has been seen in the training text of the respective language, whereas the latter improves consistency between the source and the target segmentation. If we apply BPE independently, the same name may be segmented differently in the two languages, which makes it harder for the neural models to learn a mapping between the subword units. To increase the consistency between English and Russian segmentation despite the differing alphabets, we transliterate the Russian vocabulary into Latin characters with ISO-9 to learn the joint BPE encoding, then transliterate the BPE merge operations back into Cyrillic to apply them to the Russian training text.⁵

4 Evaluation

We aim to answer the following empirical questions:

- Can we improve the translation of rare and unseen words in neural machine translation by representing them via subword units?
- Which segmentation into subword units performs best in terms of vocabulary size, text size, and translation quality?

We perform experiments on data from the shared translation task of WMT 2015. For English→German, our training set consists of 4.2 million sentence pairs, or approximately 100 million tokens. For English→Russian, the training set consists of 2.6 million sentence pairs, or approximately 50 million tokens. We tokenize and true-case the data with the scripts provided in Moses (Koehn et al., 2007). We use newstest2013 as development set, and report results on newstest2014 and newstest2015.

We report results with BLEU (*mteval-v13a.pl*), and CHRF3 (Popović, 2015), a character n-gram F_3 score which was found to correlate well with

⁴In practice, we simply concatenate the source and target side of the training set to learn joint BPE.

⁵Since the Russian training text also contains words that use the Latin alphabet, we also apply the Latin BPE operations.

on many popular image classification datasets), Adam leads to worse results than SGD with momentum (for which L_2 regularization behaves as expected).

Weight decay is equally effective in both SGD and Adam. For SGD, it is equivalent to L_2 regularization, while for Adam it is not.

Optimal weight decay depends on the total number of batch passes/weight updates. Our empirical analysis of SGD and Adam suggests that the larger the runtime/number of batch passes to be performed, the smaller the optimal weight decay.

Adam can substantially benefit from a scheduled learning rate multiplier. The fact that Adam is an adaptive gradient algorithm and as such adapts the learning rate for each parameter does *not* rule out the possibility to substantially improve its performance by using a global learning rate multiplier, scheduled, e.g., by cosine annealing.

The main contribution of this paper is to *improve regularization in Adam by decoupling the weight decay from the gradient-based update*. In a comprehensive analysis, we show that Adam generalizes substantially better with decoupled weight decay than with L_2 regularization, achieving 15% relative improvement in test error (see Figures 2 and 3); this holds true for various image recognition datasets (CIFAR-10 and ImageNet32x32), training budgets (ranging from 100 to 1800 epochs), and learning rate schedules (fixed, drop-step, and cosine annealing; see Figure 1). We also demonstrate that our decoupled weight decay renders the optimal settings of the learning rate and the weight decay factor much more independent, thereby easing hyperparameter optimization (see Figure 2).

The main motivation of this paper is to improve Adam to make it competitive w.r.t. SGD with momentum even for those problems where it did not use to be competitive. We hope that as a result, practitioners do not need to switch between Adam and SGD anymore, which in turn should reduce the common issue of selecting dataset/task-specific training algorithms and their hyperparameters.

2 DECOUPLING THE WEIGHT DECAY FROM THE GRADIENT-BASED UPDATE

In the weight decay described by Hanson & Pratt (1988), the weights θ decay exponentially as

$$\theta_{t+1} = (1 - \lambda)\theta_t - \alpha \nabla f_t(\theta_t), \quad (1)$$

where λ defines the rate of the weight decay per step and $\nabla f_t(\theta_t)$ is the t -th batch gradient to be multiplied by a learning rate α . For standard SGD, it is equivalent to standard L_2 regularization:

Proposition 1 (Weight decay = L_2 reg for standard SGD). *Standard SGD with base learning rate α executes the same steps on batch loss functions $f_t(\theta)$ with weight decay λ (defined in Equation 1) as it executes without weight decay on $f_t^{\text{reg}}(\theta) = f_t(\theta) + \frac{\lambda'}{2} \|\theta\|_2^2$, with $\lambda' = \frac{\lambda}{\alpha}$.*

The proofs of this well-known fact, as well as our other propositions, are given in Appendix A.

Due to this equivalence, L_2 regularization is very frequently referred to as weight decay, including in popular deep learning libraries. However, as we will demonstrate later in this section, this equivalence does *not* hold for adaptive gradient methods. One fact that is often overlooked already for the simple case of SGD is that in order for the equivalence to hold, the L_2 regularizer λ' has to be set to $\frac{\lambda}{\alpha}$, i.e., if there is an overall best weight decay value λ , the best value of λ' is tightly coupled with the learning rate α . In order to decouple the effects of these two hyperparameters, we advocate to decouple the weight decay step as proposed by Hanson & Pratt (1988) (Equation 1).

Looking first at the case of SGD, we propose to decay the weights simultaneously with the update of θ_t based on gradient information in Line 9 of Algorithm 1. This yields our proposed variant of SGD with momentum using decoupled weight decay (**SGDW**). This simple modification explicitly decouples λ and α (although some problem-dependent implicit coupling may of course remain as for any two hyperparameters). In order to account for a possible scheduling of both α and λ , we introduce a scaling factor η_t delivered by a user-defined procedure *SetScheduleMultiplier*(t).

Now, let's turn to adaptive gradient algorithms like the popular optimizer Adam Kingma & Ba (2014), which scale gradients by their historic magnitudes. Intuitively, when Adam is run on a loss function f plus L_2 regularization, weights that tend to have large gradients in f do not get regularized as much as they would with decoupled weight decay, since the gradient of the regularizer gets scaled

Algorithm 1 SGD with L₂ regularization and SGD with decoupled weight decay (SGDW), both with momentum

```

1: given initial learning rate  $\alpha \in \mathbb{R}$ , momentum factor  $\beta_1 \in \mathbb{R}$ , weight decay/L2 regularization factor  $\lambda \in \mathbb{R}$ 
2: initialize time step  $t \leftarrow 0$ , parameter vector  $\theta_{t=0} \in \mathbb{R}^n$ , first moment vector  $\mathbf{m}_{t=0} \leftarrow \mathbf{0}$ , schedule multiplier  $\eta_{t=0} \in \mathbb{R}$ 
3: repeat
4:    $t \leftarrow t + 1$ 
5:    $\nabla f_t(\theta_{t-1}) \leftarrow \text{SelectBatch}(\theta_{t-1})$   $\triangleright$  select batch and return the corresponding gradient
6:    $\mathbf{g}_t \leftarrow \nabla f_t(\theta_{t-1}) + \lambda \theta_{t-1}$ 
7:    $\eta_t \leftarrow \text{SetScheduleMultiplier}(t)$   $\triangleright$  can be fixed, decay, be used for warm restarts
8:    $\mathbf{m}_t \leftarrow \beta_1 \mathbf{m}_{t-1} + \eta_t \alpha \mathbf{g}_t$ 
9:    $\theta_t \leftarrow \theta_{t-1} - \mathbf{m}_t - \eta_t \lambda \theta_{t-1}$ 
10: until stopping criterion is met
11: return optimized parameters  $\theta_t$ 

```

Algorithm 2 Adam with L₂ regularization and Adam with decoupled weight decay (AdamW)

```

1: given  $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}, \lambda \in \mathbb{R}$ 
2: initialize time step  $t \leftarrow 0$ , parameter vector  $\theta_{t=0} \in \mathbb{R}^n$ , first moment vector  $\mathbf{m}_{t=0} \leftarrow \mathbf{0}$ , second moment vector  $\mathbf{v}_{t=0} \leftarrow \mathbf{0}$ , schedule multiplier  $\eta_{t=0} \in \mathbb{R}$ 
3: repeat
4:    $t \leftarrow t + 1$ 
5:    $\nabla f_t(\theta_{t-1}) \leftarrow \text{SelectBatch}(\theta_{t-1})$   $\triangleright$  select batch and return the corresponding gradient
6:    $\mathbf{g}_t \leftarrow \nabla f_t(\theta_{t-1}) + \lambda \theta_{t-1}$ 
7:    $\mathbf{m}_t \leftarrow \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t$   $\triangleright$  here and below all operations are element-wise
8:    $\mathbf{v}_t \leftarrow \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2$ 
9:    $\hat{\mathbf{m}}_t \leftarrow \mathbf{m}_t / (1 - \beta_1^t)$   $\triangleright \beta_1$  is taken to the power of  $t$ 
10:   $\hat{\mathbf{v}}_t \leftarrow \mathbf{v}_t / (1 - \beta_2^t)$   $\triangleright \beta_2$  is taken to the power of  $t$ 
11:   $\eta_t \leftarrow \text{SetScheduleMultiplier}(t)$   $\triangleright$  can be fixed, decay, or also be used for warm restarts
12:   $\theta_t \leftarrow \theta_{t-1} - \eta_t \left( \alpha \hat{\mathbf{m}}_t / (\sqrt{\hat{\mathbf{v}}_t} + \epsilon) + \lambda \theta_{t-1} \right)$ 
13: until stopping criterion is met
14: return optimized parameters  $\theta_t$ 

```

along with the gradient of f . This leads to an inequivalence of L₂ and decoupled weight decay regularization for adaptive gradient algorithms:

Proposition 2 (Weight decay $\neq$ L₂ reg for adaptive gradients). *Let O denote an optimizer that has iterates $\theta_{t+1} \leftarrow \theta_t - \alpha \mathbf{M}_t \nabla f_t(\theta_t)$ when run on batch loss function $f_t(\theta)$ without weight decay, and $\theta_{t+1} \leftarrow (1 - \lambda) \theta_t - \alpha \mathbf{M}_t \nabla f_t(\theta_t)$ when run on $f_t(\theta)$ with weight decay, respectively, with $\mathbf{M}_t \neq k\mathbf{I}$ (where $k \in \mathbb{R}$). Then, for O there exists no L₂ coefficient λ' such that running O on batch loss $f_t^{\text{reg}}(\theta) = f_t(\theta) + \frac{\lambda'}{2} \|\theta\|_2^2$ without weight decay is equivalent to running O on $f_t(\theta)$ with decay $\lambda \in \mathbb{R}^+$.*

We decouple weight decay and loss-based gradient updates in Adam as shown in line 12 of Algorithm 2; this gives rise to our variant of Adam with decoupled weight decay (**AdamW**).

Having shown that L₂ regularization and weight decay regularization differ for adaptive gradient algorithms raises the question of how they differ and how to interpret their effects. Their equivalence for standard SGD remains very helpful for intuition: both mechanisms push weights closer to zero, at the same rate. However, for adaptive gradient algorithms they differ: with L₂ regularization, the sums of the gradient of the loss function and the gradient of the regularizer (i.e., the L₂ norm of the weights) are adapted, whereas with decoupled weight decay, only the gradients of the loss function are adapted (with the weight decay step separated from the adaptive gradient mechanism). With L₂ regularization both types of gradients are normalized by their typical (summed) magnitudes, and therefore weights x with large typical gradient magnitude s are regularized by a smaller relative amount than other weights. In contrast, decoupled weight decay regularizes all weights with the same rate λ , effectively regularizing weights x with large s more than standard L₂ regularization